Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I
Segundo Cuatrimestre de 2026

Guía Práctica No. 3: Divide & Conquer / Decrease & Conquer

Esta guía práctica abarca las técnicas de diseño Divide & Conquer y Decrease & Conquer. Refiere por lo tanto a los capítulos 4 y 5 de *Introduction to the Design and Analysis of Algorithms* (Levitin 2003), que son lectura recomendada para el desarrollo de la práctica.

Parte de esta práctica cuenta con esquemas de programación y tests de asistencia a su resolución. Éstos pueden accederse a partir del classroom de github de la materia.

Esta práctica no tiene entrega formal. Su resolución es opcional.

1. Dado un arreglo `A[0..n-1]` de números reales distintos, llamaremos *inversión* a un par de valores `(A[i],A[j])` almacenados en el arreglo tales que `A[i]` $>$ `A[j]`, con `i` $<$ `j`. Diseñe, utilizando Divide & Conquer, un algoritmo que calcule el número de inversiones en un arreglo dado. Calcule el tiempo de ejecución en el peor caso de su algoritmo, e impleméntelo en Java. Por ejemplo, para el arreglo $[3, 1, 5, 2]$ la cantidad de inversiones que se realizan es 3 y para el arreglo $[4, 1, 3, 2]$ la cantidad de inversiones que se realizan es 4.

2. Diseñe usando Divide & Conquer e implemente en Java un algoritmo para resolver el problema de encontrar la subsecuencia de suma mínima, de una secuencia dada. Realice el análisis de tiempo de ejecución para el peor caso de su algoritmo. En caso de tener complejidad superior a $O(n \times \log n)$, optimice su algoritmo para conseguir tal eficiencia.

3. A cuáles de estas ecuaciones de recurrencia es posible aplicar el Teorema Maestro?

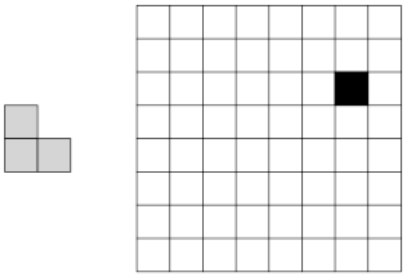
   - $T(n) = 4T(n/2) + n$
   - $T(n) = 4T(n/2) + n^2$
   - $T(n) = 4T(n/2) + n^3$
   - $T(n) = 4T(n/2) + 2^n$
   - $T(n) = 4T(n-2) + n$
   - $T(n) = 4T(n-1) + n^2$
   - $T(n) = 4T(n-4) + 1$

4. En los casos en que sea posible, aplicar el Teorema Maestro, para decidir cuál es la tasa de crecimiento de las siguientes ecuaciones de recurrencia:

   - $T(1) = 1; T(n) = 4T(\frac{n}{2}) + n$
   - $T(1) = 1; T(n) = 4T(\frac{n}{2}) + n^2$
   - $T(1) = 1; T(n) = T(\frac{n}{2}) + 1$
   - $T(1) = 1; T(n) = 2T(n-1) + 2n$
   - $T(1) = 1; T(n) = T(\frac{n}{2}) + 2^n$
   - $T(1) = 1; T(n) = 2T(\frac{n}{2}) + \log n$

5. Implemente en Java o Haskell el algoritmo para resolver el problema de, dado un conjunto de puntos del plano, encontrar el par de puntos (distintos) cuya distancia es la menor entre todos los puntos del plano, en $O(n \log n)$, discutido en *Introduction to the Design and Analysis of Algorithms.* (Levitin).

6. Un trominó (más precisamente, un trominó recto) es una pieza en forma de L formada por tres casillas adyacentes de 1×1. El problema consiste en cubrir cualquier tablero de ajedrez de $2^n \times 2^n$, con excepción de una única casilla especial ya cubierta (que puede estar en cualquier posición en la grilla), con trominós. Los trominós pueden orientarse de forma arbitraria, pero deben cubrir todas las casillas del tablero excepto la casilla inicial ya cubierta, de forma exacta y sin solaparse unas con otras.



Diseñe una solución Divide & Conquer para el problema de los trominós, e implemente su solución en Haskell.

7. Diseñe usando Divide & Conquer e implemente en Java un algoritmo para resolver el problema de dibujar la silueta de los edificios de una ciudad (skyline). Dada la ubicación y forma de n edificios rectangulares en 2-dimensiones, computar la silueta de los edificios eliminando las líneas superpuestas.

   Para el skyline que se presenta en la figura 1

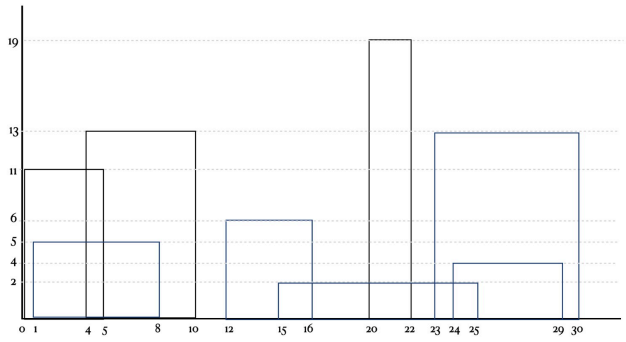


Figure 1: Skyline

La silueta queda representada por la figura 2 que se muestra debajo. Notar que la silueta del skyline se define únicamente por la lista de puntos claves marcados en la figura. Como referencia, dentro del código de base asociado a este ejercicio, pueden encontrar el test $test_example()$ que implementa este escenario.

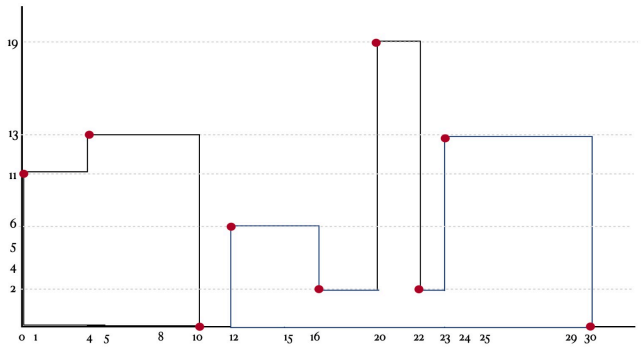


Figure 2: Skyline (Silueta)

8. Utilizando la técnica Decrease & Conquer, diseñe un algoritmo para encontrar los elementos mayor y menor de una secuencia de n enteros positivos. Implemente su algoritmo en Haskell.

9. Implemente en Java soluciones Decrease & Conquer con decremento por una constante y por un factor constante para el problema de multiplicar dos números enteros. Calcule además el número de sumas efectuadas por su algoritmo, en función del tamaño de la entrada.

   En relación a una de sus soluciones, considera que es más económico computar $n + n$ o $2 \times n$? Justifique su respuesta.

10. Utilizando la técnica Decrease & Conquer, diseñe un algoritmo para encontrar el índice del mayor elemento de una secuencia de n enteros positivos.

11. Utilizando la técnica Decrease & Conquer, diseñe e implemente en Java y Haskell un algoritmo que calcule todos los subconjuntos de un conjunto dado.

12. Utilizando la técnica Decrease & Conquer, diseñe e implemente en Java y Haskell un algoritmo que calcule todas las permutaciones de una lista.

13. Utilizando la técnica Decrease & Conquer, diseñe un algoritmo para encontrar la distancia Hamming entre dos cadenas de igual longitud.

14. Diseñe e implemente en Java y Haskell un programa que, dadas dos secuencias de caracteres, construya la subsecuencia común a ambas de longitud máxima. Se entiende por *subsecuencia* una cadena de caracteres que se deriva de la original mediante la eliminación de caracteres pero sin cambiar el orden de los caracteres en la secuencia original.